

MODULE 01 · DOMAIN 1 - FOUNDATION MODEL INTEGRATION, DATA MANAGEMENT, AND COMPLIANCE

Designing the Solution and Choosing the Model

Task 1.1 and 1.2. Requirements to architecture, model selection, invocation APIs, throughput modes and resilience.

DOMAIN 1 of 5 EXAM WEIGHT 31% MODULE 1 of 22

Model selection questions are never about which model is **best**. They are about which options survive every constraint in the scenario.

Five constraints hide in almost every design question

01

Capability

Modality, context length, reasoning depth, tool use.

02

Latency

Interactive, or can the user wait?

03

Throughput

Peak tokens per minute, and how bursty.

04

Residency

Must processing stay inside a geography?

05

Cost

Predictable spend, or lowest possible unit rate?

ON THE EXAM

Underline these five as you read. Four of the five map to a specific Bedrock feature, and the mapping is the whole of Task 1.2.

The Well-Architected Generative AI Lens

- ◆ An official AWS lens in the Well-Architected Tool lens catalogue, launched April 2025.
- ◆ Applies the six pillars across a **six-phase lifecycle**: scoping, model selection, customisation, development, deployment, continuous improvement.
- ◆ When a question mentions **consistent, repeatable architecture across teams**, this is the intended answer.
- ◆ Paired with **AWS Service Catalog** or **CloudFormation** for standardised, reusable components.
- ◆ It is a review framework, not a service. It does not enforce anything at runtime.

ON THE EXAM

Skill 1.1.3 asks for standardised technical components across deployment scenarios. That is the Lens for assessment, and Service Catalog or IaC for enforcement.

What is actually on Bedrock now

- ◆ Amazon Nova and Nova 2. Anthropic Claude. Meta Llama. Plus Mistral, Cohere, AI21, DeepSeek, Qwen, OpenAI and Stability.
- ◆ **Do not memorise model names or prices.** The catalogue turns over faster than any exam can track.
- ◆ Memorise the **shape** instead: a micro tier, a balanced tier, a frontier tier, plus embedding and image models.
- ◆ Questions describe a model by its properties, not its name: *low latency and low cost*, or *long context multimodal*.
- ◆ Amazon **Titan text** models are retired from Bedrock. Titan **embedding** models are still there and still widely used.

ON THE EXAM

The in-scope list names Amazon Titan. Treat that as meaning the embedding models, which remain current, rather than the retired text models.

Bedrock model lifecycle, and why it appears in questions

STAGE	WHAT IT MEANS	WHAT YOU MUST DO
Active	Generally available, recommended for new work	Nothing
Legacy	Superseded. An EOL date is published	Plan migration to the successor
EOL	No longer available for inference	Requests fail. You must have moved

ON THE EXAM

AWS publishes EOL dates in advance. A question describing a model that will stop working is asking about lifecycle management, not about model quality.

Converse or InvokeModel?

USE CONVERSE

The unified message API

- One request shape across every chat model
- Swap models by changing modelId only
- Native tool use, images and documents
- Guardrails attach via guardrailConfig

USE INVOKEMODEL

The provider-native API

- You send and parse the provider's own JSON
- Required for embeddings and image models
- Required for provider fields Converse omits
- Legacy integrations not yet migrated

THE TIEBREAKER

Same models, same token price. If the question mentions swapping models without rewriting code, it is describing Converse.

The Bedrock runtime API surface

OPERATION	SHAPE	USE IT FOR
<code>Converse</code>	Unified, synchronous	Chat, tool use, multimodal. The default
<code>ConverseStream</code>	Unified, streamed	Anything a human watches appear
<code>InvokeModel</code>	Native, synchronous	Embeddings, image generation, native fields
<code>InvokeModelWithResponseStream</code>	Native, streamed	Streaming where native payloads are required
<code>InvokeModelWithBidirectionalStream</code>	Native, full duplex	Real-time speech in and out
<code>AsyncInvoke</code>	Fire and forget	Long jobs writing results to S3

ON THE EXAM

Two axes only: unified or native, and synchronous, streamed or asynchronous. Locate any option on that grid and the answer usually falls out.

The parameters, and what they actually control

- ◆ **temperature** flattens or sharpens the probability distribution. Low means deterministic and repeatable.
- ◆ **topP** samples from the smallest set of tokens whose probabilities sum to P. **topK** caps that set by count.
- ◆ **maxTokens** caps *output* length only. It does not shrink your input bill.
- ◆ **stopSequences** ends generation on a string. Useful for structured output.
- ◆ Tuning these **cannot** stop hallucination. That is a grounding problem, covered in module 11.

ON THE EXAM

Adjust temperature or topP, not both. Moving both at once makes the effect of either impossible to attribute.

For extraction, classification and anything feeding a downstream system, the answer is **low temperature**. Creativity parameters are for prose, not pipelines.

A media company runs a customer-facing assistant on Amazon Bedrock through a Lambda function behind API Gateway. The assistant answers questions in long, detailed paragraphs. Users complain the assistant feels slow and often abandon the page before an answer appears. CloudWatch shows median model latency of 4.1 seconds and no throttling. The team has already reduced maxTokens with no improvement in complaints, and the answers are now criticised as incomplete.

What should the team change to improve the perceived responsiveness?

- ☐ A Switch from Converse to ConverseStream and stream tokens to the browser as they are generated
- ☐ B Move the workload to Provisioned Throughput to guarantee capacity
- ☐ C Enable a global cross-Region inference profile to reduce latency
- ☐ D Increase the Lambda function memory allocation and enable provisioned concurrency

◆ PAUSE AND DECIDE

What should the team change to improve the perceived responsiveness?

- A** Switch to ConverseStream and stream tokens to the browser
Time to first token drops to a fraction of a second. Total latency is unchanged, but the wait stops being blank.
- B** Move to Provisioned Throughput
Provisioned capacity fixes throttling. The scenario explicitly states there is none, so this buys reserved capacity nobody needs.
- C** Enable a global cross-Region inference profile
Cross-Region inference is a throughput mechanism, not a latency one. It can add a network hop rather than remove one.
- D** Increase Lambda memory and provisioned concurrency
Attacks cold starts, worth milliseconds. The four seconds is the model generating, and Lambda cannot speed that up.

THE SIGNAL

"No throttling" eliminates B. "Users abandon before an answer appears" is a perceived-latency complaint, not a total-latency one. Streaming fixes perception without touching either.

There are now **six** ways to buy inference on Bedrock, not two. Most study material is still on the old list, and the differences are exam-relevant.

How you can pay for Bedrock inference

MODE	HOW IT WORKS	CHOOSE IT WHEN
Standard	Default on-demand tier	Traffic is unpredictable or you are prototyping
Priority	serviceTier=priority. 75% premium	Interactive and must stay fast under load
Flex	serviceTier=flex. 50% discount	Synchronous but tolerant of extra latency
Batch	Async job, S3 in and S3 out. 50% discount	Bulk work with no one waiting
Reserved	serviceTier=reserved. Fixed monthly price	Mission critical, predictable tokens per minute
Provisioned Throughput	Model units billed hourly	Custom or fine-tuned models

ON THE EXAM

The first three and Reserved are one parameter on the same call. Batch and Provisioned Throughput change your architecture.

Reserved tier or Provisioned Throughput?

RESERVED TIER

Reserve tokens per minute

- Fixed price per 1,000 tokens per minute
- 1 or 3 month term, billed monthly
- Targets 99.5% model response uptime
- Overflows to Standard when you exceed it

PROVISIONED THROUGHPUT

Reserve model units

- Hourly rate per model unit
- No commitment, 1 month or 6 month
- The only way to serve a custom model
- Idle capacity still bills in full

THE TIEBREAKER

Custom or fine-tuned model in the scenario? Provisioned Throughput, no argument.
Base model with steady predictable load? Reserved is the newer and usually better fit.

Reading the money constraint

- ◆ "No one is waiting" plus bulk volume equals **Batch**. Nightly jobs, backfills, bulk embedding.
- ◆ "Tolerates some latency" but still synchronous equals **Flex**. Evaluations, summarisation, background agent steps.
- ◆ "Cannot be throttled" plus **steady, predictable** volume equals **Reserved**.
- ◆ "Bursty and unpredictable" equals **Standard**, plus cross-Region inference for headroom.
- ◆ Reserving capacity for bursty traffic is a classic distractor. You pay for the peak and idle through the trough.

ON THE EXAM

The word 'bursty' is a reservation-killer. The word 'predictable' or 'sustained' is a reservation-enabler. Scenarios include one of them deliberately.

A logistics company must summarise 2.4 million historical shipping exception reports so analysts can search them. The reports sit in Amazon S3. Summaries are needed within a week and no user or system is waiting on any individual result.

A pilot on 5,000 reports using synchronous on-demand invocation worked correctly but the finance team has rejected the projected cost of the full run. The team must reduce cost substantially and is willing to change how the job runs, but not to change models or degrade summary quality.

Which approach meets the requirements at the lowest cost?

- | | |
|---|--|
| A Keep synchronous invocation but set serviceTier to flex, and run the job across several days | B Submit the reports as a Bedrock batch inference job reading from and writing to Amazon S3 |
| C Purchase Provisioned Throughput model units for one month and run the job at full rate | D Switch to a smaller model and keep synchronous on-demand invocation |

◆ PAUSE AND DECIDE

Which approach meets the requirements at the lowest cost?

- A** Synchronous with serviceTier=flex
Also 50% off, and defensible. But it keeps 2.4 million synchronous calls the team must orchestrate and retry themselves.
- B** Bedrock batch inference job via S3
50% discount, data already in S3, and Bedrock owns the orchestration and retries. Nothing is waiting, which is exactly Batch's precondition.
- C** Provisioned Throughput for one month
Buys guaranteed capacity at a premium hourly rate. The job has no latency requirement to protect, so this raises cost.
- D** Switch to a smaller model
Directly violates the stated constraint that quality must not degrade and models must not change.

THE SIGNAL

A and B both give 50%. Batch wins on operational overhead: the data is already in S3 and no result is time-sensitive. When two options match on price, the tiebreaker is who runs the loop.

What actually happens when you call an inference profile

01

You call

Your app invokes a profile ID, not a bare model ID.

02

Bedrock routes

It picks a destination Region with available capacity.

03

Inference runs

Compute happens in that destination Region.

04

Response returns

Encrypted, back to your source Region.

ON THE EXAM

Nothing is stored in the destination Region. Logs, knowledge bases and configuration all stay in the source Region. CRIS moves compute, not data at rest.

Geographic or global inference profile?

GEOGRAPHIC PROFILE

Routes inside one geography

- Prefixes us., eu., apac., jp., au.
- An EU request only reaches EU Regions
- The profile type built for residency rules
- Higher throughput than single-Region

GLOBAL PROFILE

Routes worldwide

- Prefix global.
- Any supported commercial Region
- The highest throughput available
- Often a lower per-token rate

THE TIEBREAKER

Any mention of a regulator, sovereignty, or data staying in a jurisdiction forces the geographic profile. Global is cheaper and faster and still wrong.

A European insurer is deploying a claims assistant built on Amazon Bedrock. Its regulator requires that all processing of policyholder data occurs within the European Union, and the compliance team must be able to demonstrate this.

Traffic is highly variable, peaking sharply on Monday mornings. A single-Region deployment in eu-west-1 has been returning throttling errors at peak. The team wants more headroom without breaching the residency requirement, and wants a control that prevents a developer from accidentally routing outside the EU.

Which combination should the insurer implement?

- ☐ A Use a global cross-Region inference profile, and enable model invocation logging to prove where requests ran
- ☐ B Use an EU geographic inference profile, and add an SCP denying `bedrock:InvokeModel` on non-EU inference profile ARNs
- ☐ C Deploy independent Bedrock stacks in three EU Regions with Route 53 latency-based routing between them
- ☐ D Stay single-Region in eu-west-1 and request a service quota increase for the model

Which combination should the insurer implement?

- A

 Global profile plus invocation logging
Global routes to any commercial Region worldwide. Logging proves the breach after it happens; it does not prevent it.
- B

 EU geographic profile plus an SCP on non-EU profile ARNs
The profile bounds routing to the EU, and the SCP makes the boundary enforceable rather than conventional.
- C

 Three EU stacks behind Route 53
Would work and stays in the EU, but rebuilds cross-Region inference by hand. Far more operational overhead for the same outcome.
- D

 Single Region plus a quota increase
A quota raises the in-Region ceiling but leaves you inside one Region's capacity. Sharp Monday peaks are what CRIS exists for.

THE SIGNAL

"Prevents a developer from accidentally routing outside the EU" is asking for a preventive control, not a detective one. Logging detects. An SCP prevents.

Designing for the day a model or Region misbehaves

- ◆ **Retry with exponential backoff and jitter.** The SDK does this. Throttling is normal, not exceptional.
- ◆ **Circuit breaker.** Step Functions or application state stops hammering a failing dependency and fails fast.
- ◆ **Model fallback.** Frontier model unavailable, drop to a smaller one rather than returning an error.
- ◆ **Graceful degradation.** Serve a cached answer, a template, or a queued promise instead of a failure page.
- ◆ **Cross-Region inference** covers regional capacity. It does not cover a bad deployment of your own code.

ON THE EXAM

Exam wording: 'continuous operation during service disruptions' means fallback and degradation, not more capacity. Capacity answers a throughput problem, not an availability one.

Changing models without changing code

- ◆ **AWS AppConfig** holds the model ID and inference parameters as dynamic configuration.
- ◆ Lambda fetches config at runtime. Change the model, no deployment, no code change, **no rollback risk**.
- ◆ AppConfig adds **gradual rollout** and **automatic rollback on a CloudWatch alarm**.
- ◆ Pairs with **Converse**, since only modelId changes. With InvokeModel you would also swap payload shapes.
- ◆ Hardcoding the model ID, or holding it in a Lambda environment variable, both require redeployment.

ON THE EXAM

"Switch models without modifying code" is almost a literal quotation from Skill 1.2.2, and AppConfig is the service it names. It appeared 10 times in the practice bank.

A SaaS company invokes Amazon Bedrock from Lambda functions using the Converse API. The model ID is currently a Lambda environment variable.

New models appear on Bedrock every few weeks and the team wants to evaluate and adopt them quickly. They also want to roll a new model out to a small percentage of traffic first, and revert automatically if the error rate rises. Any change must not require a code deployment, and platform engineering has asked for the solution with the least operational overhead.

What should the company implement?

- ☐ A Store the model ID in AWS AppConfig, retrieve it at runtime, and use AppConfig deployment strategies with a CloudWatch alarm for rollback
- ☐ B Store the model ID in AWS Secrets Manager and read it on each invocation
- ☐ C Publish Lambda aliases per model and shift traffic with weighted alias routing in CodeDeploy
- ☐ D Store the model ID in an Amazon DynamoDB table and build a Lambda admin function to update it

◆ PAUSE AND DECIDE

What should the company implement?

- A

 AppConfig with deployment strategies and alarm rollback
Purpose-built for exactly this. Runtime config, gradual rollout and automatic rollback are all native features, not things you build.
- B

 Secrets Manager
Would technically work but a model ID is not a secret. No gradual rollout, no automatic rollback, and you pay per secret.
- C

 Lambda aliases and weighted CodeDeploy routing
Gives gradual rollout and rollback, but shifting traffic between code versions means publishing a version. That is a deployment.
- D

 DynamoDB plus an admin Lambda
Solves it, and you now own a config service. Gradual rollout and rollback logic are all yours to write and maintain.

THE SIGNAL

Three options work. 'Least operational overhead' plus 'no code deployment' plus 'automatic rollback' names three AppConfig features at once. When one service matches every clause, stop looking.

Where this module is newer than the exam

- ◆ Service tiers are recent. Priority and Flex arrived November 2025, Reserved shortly after. Question banks may not have caught up.
- ◆ If a question offers only on-demand, batch and provisioned throughput, **answer within that set**. Do not look for a tier that is not on the page.
- ◆ **Reserved tier is not self-service**. You request it through your AWS account team, which is why it is rare in scenarios.
- ◆ **Latency-optimised inference** is still a preview feature on selected models. Useful to recognise, unlikely to be the sole right answer.
- ◆ The underlying decisions have not changed: predictable or bursty, waiting or not waiting, bounded geography or not.

ON THE EXAM

This is the reverse of the module 0 warning. There, the exam listed services that had gone. Here, features exist that the exam may not yet know about.

A bank has fine-tuned a foundation model on its internal credit policy documents. The fine-tuned model must serve an internal underwriting tool during business hours, at a steady and well-understood request rate.

During a pilot the team found that invoking the fine-tuned model with a standard on-demand call returned a `ValidationException`. An architect has proposed simply retrying with backoff. Cost is a concern but correctness is not negotiable.

What is the correct way to serve this model, and why did the pilot fail?

- | | |
|--|---|
| <p>A Create a Provisioned Throughput deployment for the custom model and invoke it by that deployment ARN</p> | <p>B Set <code>serviceTier</code> to <code>reserved</code>. The <code>Reserved</code> tier is required for fine-tuned models</p> |
| <p>C Retry with exponential backoff. The <code>ValidationException</code> indicates transient capacity pressure</p> | <p>D Invoke the model through a global cross-Region inference profile to find available capacity</p> |

◆ PAUSE AND DECIDE

What is the correct way to serve this model, and why did the pilot fail?

- A

 Provisioned Throughput deployment, invoked by its ARN
A custom model must be deployed before it serves anything, and is then called by the deployment ARN. The pilot passed a base model ID, which no deployment backs.
- B

 serviceTier=reserved
Reserved is a service tier for base models on the on-demand path. It does not make a custom model invocable.
- C

 Retry with exponential backoff
Reads the error wrongly. ValidationException means the request was malformed or unsupported. Retrying an invalid request returns the same error forever.
- D

 Global cross-Region inference profile
CRIS routes base models across Regions. A custom model exists only where you customised it, so there is nothing to route to.

THE SIGNAL

ThrottlingException means slow down and retry. ValidationException means the request itself is unsupported, so retrying is pointless. Module 2 covers the on-demand custom model deployment option, which is newer and narrower than Provisioned Throughput.

The five things to carry forward

- ◆ Read for the **five constraints**. Capability, latency, throughput, residency, cost.
- ◆ **Converse** for portability and chat. **InvokeModel** for embeddings, images and native fields. Stream when a human watches.
- ◆ **Batch** when nobody waits. **Flex** when they will wait a bit. **Reserved** when steady and critical. **Provisioned Throughput** when the model is custom.
- ◆ **Geographic** inference profile for residency. **Global** for maximum throughput. Never global when a regulator is mentioned.
- ◆ **AppConfig** is the answer to "change the model without changing code".

ON THE EXAM

Module 2 takes the next step: the model you chose is not quite good enough. Prompt, retrieve, fine-tune, or distil?